Lab 2: Sexism Identification in Twitter

Session 5. Large Language Models: Prompting and In-context Learning

Goal

This lab session aims to help students understand and practice using large language models (LLMs), particularly Transformer-based models utilizing the LLaMA architecture. The second objective is the application of these models to sexism identification tasks, specifically employing in-context learning techniques, including zero-shot and few-shot approaches through the HuggingFace APIs for the three subtasks proposed in *"EXIST2025: sEXism Identification In Social neTworks"*.

Bibliography

- Jurafsky D., and Martin J. 2024. *Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition with Language Models*, 3rd edition. Online manuscript released August 20, 2024. https://web.stanford.edu/~jurafsky/slp3.
- Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., & Sutskever, I. (2019). *Language Models are Unsupervised Multitask Learners*. OpenAI blog 2019.
- Tom B. Brown, et al. 2020. *Language models are few-shot learners*. In Proceedings of the 34th International Conference on Neural Information Processing Systems (NIPS '20). Curran Associates Inc., Red Hook, NY, USA, Article 159, 1877–1901.

- Touvron, H. et al (2023). LLaMA: Open and Efficient Foundation Language Models. CoRR abs/2302.13971. 2023
- Touvron, H. et al (2023). LLaMA 2: Open Foundation and Fine-Tuned Chat Models. ArXiv, abs/2307.09288. 2023
- https://huggingface.co/docs/transformers/v4.47.0/en/model_doc/auto#transformers.AutoModelForCausalLM.

Large Language Models

The LLMs are a kind of artificial intelligence that excels at processing, understanding, and generating human language. These models are based on deep learning architectures and are trained on massive amounts of text data to recognize complex patterns in language. LLMs exhibit remarkable performance on natural language tasks because of the knowledge they learn in pretraining. They have been especially transformative for tasks where we must produce text, like summarization, machine translation, question answering, or chatbots.

LLMs are based on the Transformers architecture studied in the previous session but introduce some differences in the training and sampling strategies. Distinct BERT and BERT-based architecture, which relied on predicting the masked tokens, LLMs use conditional generation, which is the task of generating text conditioned on an input piece of text. We give the LLM an input piece of text, generally called a **prompt**, and then have the LLM continue generating text token by token, conditioned on the prompt.

The fact that transformers have such long contexts (many thousands of tokens) makes them very powerful for conditional generation because they can look back so far into the prompting text.

In recent years, leading technology companies and research institutions have significantly contributed to the landscape of large language models by developing and releasing diverse models through various access strategies. Major tech giants like OpenAI, Meta AI, Google, Anthropic, and others have pioneered different approaches to model development, ranging from fully private proprietary models to open-source and partially accessible models. In this lab session, we focused on GPT and LLaMA models, paying attention to LLaMA models, which can be accessible for academic purposes using free access.

Generative Pre-trained Transformer 2 (GPT-2)

It is a Transformer-based language model pretrained on a large corpus of English data in a self-supervised fashion. It contains 1.5 billion parameters and is designed to predict the next word in a sequence, enabling it to generate coherent and contextually relevant text. It was developed by OpenAI in 2019. The model can handle a maximum input length of 1,024 tokens. GPT-2 was the first model to demonstrate convincing zero-shot task performance. Primarily designed for the English language.

Generative Pre-trained Transformer 3 (GPT-3)

An advanced iteration of the GPT series, featuring 175 billion parameters. It uses dense and sparse attention in transformer layers and can handle a maximum input length of 4,096 tokens. The model was released in 2020 by OpenAI. GPT-3 showed a dramatic improvement in few-shot and zero-shot learning [1] capabilities. Moreover, it demonstrated the ability to perform complex tasks with minimal task-specific fine-tuning.

Generative Pre-trained Transformer 4 (GPT-4)

OpenAI's latest model, capable of processing both text and image inputs. It shows impressive performance on various tasks, including professional exams, but its architecture and training details remain undisclosed. It was released in 2023 by OpenAI. The model used an enhanced multi-modal transformer architecture, but the exact parameter count is not publicly disclosed still. GPT-4 also expands on the maximum input length compared to previous iterations, increasing it to a maximum of 32,768 tokens. The latest version of gpt-4 (GPT-4-Turbo) can handle inputs of 128 000 tokens. GPT-4 significantly improved reasoning, multimodal capabilities, and reduced hallucination compared to previous models.

Large Language Model Meta AI[2] (LLaMA)

[1] Zero-shot and few-shot learning are discussed in the next sections.

[2] https://arxiv.org/abs/2302.13971

It is an auto-regressive language model developed by Meta AI, available in sizes ranging from 7B to 65B parameters. It is designed for research purposes and can be used for tasks like report drafting, content generation, and developing AI assistants. The model, like other LLMs, works by taking a sequence of words as an input and predicts a next word to recursively generate text. LLaMA uses a Transformer decoder with efficient attention mechanisms. The model demonstrated robust performance with smaller model sizes. Access to the model is only available to researchers, government affiliates, and those in academia, and only after applying to Meta AI. Primarily designed for English language tasks.

Large Language Model Meta AI[3], 2nd Generation (LLaMA 2)

LLaMA 2 was developed by Meta AI and released in July 2023, representing a significant evolution from the original LLaMA model, introducing substantial improvements in performance, accessibility, and training methodology. The model comes in three parameter sizes (7B, 13B, and 70B), featuring a more comprehensive training dataset of 2 trillion tokens, 40% larger than the original LLaMA. Key advancements include enhanced reasoning capabilities, reduced hallucination, and improved contextual understanding. While the original LLaMA was primarily a research model with restricted access, LLaMA 2 introduces specialized chat models fine-tuned for

[3] https://arxiv.org/abs/2307.09288

conversational interactions, incorporating Reinforcement Learning from Human Feedback (RLHF) to improve alignment and safety. The model demonstrates superior performance across various benchmarks, with more sophisticated attention mechanisms and a refined transformer architecture that enables more coherent and contextually appropriate text generation. Additionally, LLaMA 2 provides more transparent licensing, making it more accessible to researchers and developers than the strictly controlled access of the original LLaMA model. Expanded language support compared to original LLaMA, but 90% of the training data is in English.

Large Language Model Meta AI[4], 3rd Generation (LLaMA 3)

LLaMA 3 is Meta AI's latest open-source large language model, designed to rival the capabilities of GPT-4. It comes in 8B and 70B parameters, and version LLaMA 3.1 has a size of 405B, each with base (pre-trained) and instruct-tuned versions. LLaMA 3 enhanced the performance in language nuances, contextual understanding, and complex tasks compared to previous versions, also it improved reasoning abilities, code generation, and instruction following. LLaMA3 used a new tokenizer with an expanded vocabulary of 128,256 tokens, allowing more efficient text encoding. Moreover, the architecture used Grouped Query Attention (GQA) [5] for

[4] https://arxiv.org/abs/2307.09288

[5] Grouped Query Attention (GQA) is an efficient attention mechanism for large language models that balances the quality of multi-head attention (MHA) with the speed of multi-query attention (MQA).

improved inference efficiency. LLaMA 3 increased its ability to handle sequences of up to 128,000 tokens and included multilingual support for over 100 languages, including English, Spanish, French, German, Chinese, and more. LLaMA 3 excels in various natural language processing tasks such as text generation, conversation, summarization, and translation. It aims to provide an open-source alternative to GPT-4 capabilities while offering greater flexibility for researchers to customize and experiment. The continued LLaMA 3 development process included multimodal extensions to the models; enabling image recognition, video recognition, and speech understanding capabilities. The resulting models are not yet being broadly released as they are still under development.

LLMs' Prompts

A **prompt** is a text string that a user issues to a language model to get the model to do something useful. In prompting, the user's prompt string is passed to the LLM, which iteratively generates tokens conditioned on the prompt. Thus, the prompt creates a context that guides a LLM to generate useful outputs to achieve some user goal. The process of finding effective prompts for a task is known as **prompt engineering**.

In this session, we explore the capabilities of LLMs by practicing task-oriented interactions through prompt engineering. We learn to use prompts that guide these models to perform language processing like the three subtasks proposed in EXIST2025.

The initial step to prompt an LLM model involves installing the necessary packages and loading the selected model. In this session, we leverage the APIs provided by HuggingFace and employ the LLaMA model. This entails setting up a suitable environment with the required libraries, such as transformers and torch. Subsequently, we can utilize the AutoModelForCausalLM class from the transformers library to load the LLaMA model. This class facilitates loading various pre-trained language models, including LLaMA, from the HuggingFace model hub. Following these steps, we establish the framework for interacting with the LLaMA model and generating text based on provided prompts.

Installing the required dependencies

```
!pip install transformers --upgrade
!pip install datasets accelerate
!pip install evaluate
```

A user access token is typically needed to access models and datasets on the Hugging Face Hub. This token serves as a credential that allows you to authenticate your identity and authorize access to resources. It is particularly crucial when working with non-entirely free models or datasets that require specific permissions. You can generate a user access token through your HuggingFace account settings. Once obtained, you can set it using the function *login ()* in the code. This ensures that

your requests to the Hugging Face API are correctly authenticated, enabling you to download, upload, and utilize the desired resources.

EXAMPLE 1: Setting the authentication access token

```
from huggingface_hub import login
# Authenticate with your HuggingFace token

login(token='XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX')
```

The second step is loading the LLaMA model on which we will work. Due to the space restriction, we used the LLaMA 2 model with 7B parameters. Moreover, due to our interest in instructing the model for recognizing sexism in the text, we considered the version optimized for dialogue use cases *"meta-llama/Llama-2-7b-chat-hf".*

EXAMPLE 2: Loading the LLaMA 2 model for conditioned generation

```
import torch
from transformers import AutoTokenizer, AutoModelForCausalLM

model_path = "meta-llama/Llama-2-7b-chat-hf"
tokenizer = AutoTokenizer.from_pretrained(model_path,
token="XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX")
model = AutoModelForCausalLM.from_pretrained(
model_path,
torch_dtype=torch.float16,
device_map='auto'
)
```

Notice that in the tokenizer also can be specified the authentication access token using the parameter *token*. The parameter *torch_dtype=torch.float16* sets the data type to half-precision floating-point numbers, which can significantly reduce memory usage, especially for large models. The parameter *device_map='auto'* maps the model to available GPUs or CPUs, optimizing its usage.

To use a different model, all you need to do is change the model_path to the desired model's repository name. Hugging Face's AutoModelForCausalLM and AutoTokenizer classes are designed to handle a wide range of models, making the process of switching between them quite straightforward.

In Example 3, we use a static prompt to determine if the tweet "*i didntmean that hes not a whore hes an empowered woman who can do whatever he wants with her body im so sorry*" is sexist or not. This tweet is an example of sexist tweets selected from the development set of EXIST2025. Finally, we give the prompt to the language models and ask to generate the 32 tokens.

EXAMPLE 3: Simple and fixed prompt to determine whether the tweets is sexist or not

```
def simple_prompting_model(model, tokenizer, prompt):
    inputs = tokenizer(prompt, return_tensors="pt").to(model.device)
    # Generate responses on pretrained model
    outputs = model.generate(
        **inputs,
```

```
        max_new_tokens=32, #Specify the new tokens that must be generated
        num_return_sequences=1, # Determines the number of different
sequences the model should generate
        temperature=0.8 # Controls the randomness of the generated text. A
higher temperature leads to more diverse and creative outputs, while a lower
temperature results in more focused and deterministic outputs.
    )

    response = tokenizer.decode(outputs[0], skip_special_tokens=True)
    return response

##########################################################################
sexist_tweets="i didntmean that hes not a whore hes an empowered woman who
can do whatever he wants with her body im so sorry"
prompt=f"Input: Determine if the tweet {sexist_tweets} contains sexist
language or not. Output:"
simple_prompting_model(model, tokenizer, prompt)
```

Model Output

Input: *Determine if the tweet i didntmean that hes not a whore hes an empowered woman who can do whatever he wants with her body im so sorry contains sexist language or not.* **Output**: ***Yes****, the tweet contains sexist language. Explanation: The term "whore" is a derogatory and offensive term that is often*

As the model generates "*Yes*" in response to the Output statement, it indicates that the tweet is sexist. While this is an interesting finding, practical sexism identification requires addressing several challenges in the **prompts design** and the **post-processing the model response**.

If we want to solve general tasks like sexism identification tasks, we do not want to have to create a new prompt each time we do the task. Instead, the first step in prompting is to design one or more **templates**: task-specific prompting text along with slots for the input being processed. Also is recommended to include specifications about the output format.

In summary, we adapt our task for an LLM by carefully designing prompts that enable the model to generate contextually appropriate responses through a clear, structured input format:

- For a given task, develop a task-specific template that has a free parameter for the input text.
- Given that input and the task-specific template, the input is used to instantiate a **filled prompt** that is then passed to a pretrained LLM.
- Autoregressive decoding is then used to generate a sequence of token outputs.
- The output of the model can either be used directly as the desired output (as in the case of naturally generative tasks such as translation or summarization), or a task-appropriate answer can be extracted from the generated output (as in the case of the sexism identification task).

In-Context Learning

Prompts get language models to generate text, but they also can be viewed as a learning signal, because these demonstrations can help language models learn to perform novel tasks. For this reason, we also refer to prompting as in-context learning—learning that

improves model performance or reduces some loss but does not involve gradient-based updates to the model's underlying parameters. It is often possible to improve a prompt by including the task description and some labeled examples in the prompt template. We call such examples **demonstrations**. The task of prompting with examples is sometimes called **few-shot** prompting, as contrasted with **zero-shot** prompting which means instructions that do not include labeled examples even when they can include the task description.

Zero-Shot Learning

Zero-shot learning is a technique where a model can perform a task without having been explicitly trained on that specific task. In the context of LLMs, this means the model can understand and complete a task using only a textual description of the task, without prior fine-tuning or specific training examples.

Some recommended aspects to include in the prompt for zero-shot are:

1. Task Specification: Clear, explicit description of the task including precise instructions about what needs to be done in precise and detailed language.
2. Context Provision: Relevant background information, including contextual details that help the model understand the task. Also, any specific constraints or requirements to be satisfied.

3. Role Definition: It is optional but helpful to define a specific role or persona for the model. It helps frame the response appropriately.
4. Output Structure: Specify the desired response format and the number of outputs

Examples 3, 4, and 5 illustrate how to perform zero-shot learning for subtasks 1, 2, and 3 in EXIST2025, respectively. In each example the LLaMA 2 model (*meta-llama/Llama-2-7b-chat-hf*) is used as language models. The examples are different with respect to how the prompt is filled in to address the task. The second difference is in how the model's output is parsed to obtain the predicted labels for each tweet. These examples serve as a starting point for understanding how Large Language Models can be applied to sexism identification tasks through zero-shot learning approaches. Additional work should focus on refining prompt strategies and developing more robust parsing mechanisms to improve classification effectiveness.

EXAMPLE 4: Using zero-shot to sexism identification

```
def output_postprocessing_incontext_zero_s1(output):
   outputp=output.rsplit("Output: ", 1)[-1].strip()
   for line in outputp.split('\n'):
       line=line.strip()
       if line!="":
            if line.upper().startswith("YES"): return "YES"
            if line.upper().startswith("NO"): return "NO"
   return "UNK"


def incontext_zero_pipeline_task1(model, tokenizer, devData, testData, postprocess, **params):
   role= "in social psychology and linguistics with vast experience analyzing social media
content and discriminative and harmful language"
   task ="""Sexist identification task is a binaty text classification task which aim at
determining
   whether or not a given tweet expresses ideas related to sexism in any of the three forms: it
is sexist itself,
   it describes a sexist situation in which discrimination towards women occurs, or criticizes
a sexist behaviour.
   The tweet is sexist (YES) or describes or criticizes a sexist situation. Not sexist. The
tweet is not sexist (NO),
```

```
   nor it describes or criticizes a sexist situation."""
   output= "The output must be YES/NO."
   context=None


   ntokens=params.get("max_new_tokens", 8)
   nseq=params.get("num_return_sequences", 1)
   temp=params.get("temperature", 0.7)


   idqueries= devData[0].tolist()
   textqueries= devData[1].tolist()
   labelqueries=devData[2].tolist()


   predictions=[]
   for i in range(len(textqueries)):
       query=textqueries[i]
       filledPrompt=create_incontext_zero_prompt(task, query, context=None, role=role,
output=output)
       response= perform_incontext_classification(model, tokenizer, filledPrompt, ntokens,
nseq, temp)
       pred=postprocess(response)
       if pred in ["YES", "NO"]:
           predictions.append(pred)
       else:
           predictions.append("NO")
       print("ID: ", idqueries[i], "Ground Truth:", labelqueries[i], "Predicted: ", pred)


   # Create dev output DataFrame
   dev_df = pd.DataFrame({ 'id': idqueries,  'label': predictions, "tweet": textqueries,
"test_case": ["EXIST2024"]*len(predictions) })
   dev_df.to_csv('sexism_dev_predictions_task1.csv', index=False)
   print("Prediction TASK1 completed. Results saved to sexism_dev_predictions_task1.csv")


   #Computing the quality measure for the subtask 1
   f1= f1_score(labelqueries, predictions, labels=None, pos_label="YES", average='binary')
   print(f"\nF1-Score Sexism: {f1}\n\n")


   idqueries= testData[0].tolist()
   textqueries= testData[1].tolist()
   test_predictions=[]
   for i in range(len(textqueries)):
       query=textqueries[i]
       filledPrompt=create_incontext_zero_prompt(task, query, context=None, role=role,
output=output)
       response= perform_incontext_classification(model, tokenizer, filledPrompt,  ntokens,
nseq, temp)
       pred=postprocess(response)
       if pred in ["YES", "NO"]:test_predictions.append(pred)
       else: test_predictions.append("NO")
       print("ID: ", idqueries[i],  "Predicted: ", pred)
       # Create submission DataFrame
   submission_df = pd.DataFrame({'id': idqueries, 'label': test_predictions, "test_case":
["EXIST2024"]*len(test_predictions) })
   submission_df.to_csv('sexism_predictions_task1.csv', index=False)
   print("Prediction TASK1 completed. Results saved to sexism_predictions_task1.csv")


params=dict()
EnTrainTask1, EnDevTask1, EnTestTask1 =reader.get(lang="EN", subset="TRAIN", task="task1"),
reader.get(lang="EN", subset="DEV", task="task1"),  reader.get(lang="EN", subset="TEST",
task="task1"),
incontext_zero_pipeline_task1(model, tokenizer, EnDevTask1, EnTestTask1,
output_postprocessing_incontext_zero_s1)
```

EXAMPLE 5: Using zero-shot to source intention identification

```
from sklearn.metrics import f1_score
import pandas as pd


def output_postprocessing_incontext_zero_s2(output):
   outputp=output.rsplit("Output: ", 1)[-1].strip()
   for line in outputp.split('\n'):
       line=line.strip()
       if line!="":
            if line.upper().startswith("DIRECT"): return "DIRECT"
            if line.upper().startswith("REPORTED"): return "REPORTED"
            if line.upper().startswith("JUDGEMENTAL"): return "JUDGEMENTAL"
   return "UNK"


def incontext_zero_pipeline_task2(model, tokenizer, devData, testData, postprocessing,
**params):


   role= "in social psychology and linguistics with vast experience analyzing social media
content and discriminative and harmful language"
   task ="""Sexism Source Intention in tweets is a three-class classification task aims to
categorize
   the sexist tweets according to the intention of the author. This distinction allows us to
differentiate
   sexism that is actually taking place online from sexism which is being suffered by women in
other situations
   but that is being reported in social networks with the aim of complaining and fighting
against sexism.
   The task includes the following ternary classification of tweets.


   REPORTED: the intention is to report and share a sexist situation suffered by a woman or
women.
   JUDGEMENTAL the intention is to condemn and criticise sexist situations or behaviours.
   DIRECT: The intention is to write a message that is sexist by itself or incites to be
sexist.
    """
   output= "The output is a single label: REPORTED/JUDGEMENTAL/DIRECT."
   context=None


   ntokens=params.get("max_new_tokens", 8)
   nseq=params.get("num_return_sequences", 1)
   temp=params.get("temperature", 0.7)


   idqueries= devData[0].tolist()
   textqueries= devData[1].tolist()
   labelqueries=devData[2].tolist()


   predictions=[]
   for i in range(len(textqueries)):
       query=textqueries[i]
       filledPrompt=create_incontext_zero_prompt(task, query, context=None, role=role,
output=output)
       response= perform_incontext_classification(model, tokenizer, filledPrompt, ntokens,
nseq, temp)
       pred=postprocessing(response)
       if pred in ["DIRECT", "REPORTED", "JUDGEMENTAL"]: predictions.append(pred)
```

```
        else: predictions.append("DIRECT")
        print("ID: ", idqueries[i], "Ground Truth:", labelqueries[i], "Predicted: ", pred)


    # Create dev output DataFrame
    dev_df = pd.DataFrame({
        'id': idqueries,
        'label': predictions,
        "tweet": textqueries,
        "test_case": ["EXIST2024"]*len(predictions) })
    dev_df.to_csv('sexism_dev_predictions_task2.csv', index=False)
    print("Evaluation TASK2 completed. Results saved to sexism_dev_predictions_task2.csv")


    #Computing the quality measure for the subtask 2
    f1= f1_score(labelqueries, predictions, labels=None, average='macro')
    print(f"\nMacro Average F1-Score Sexism: {f1}\n\n")


    idqueries= testData[0].tolist()
    textqueries= testData[1].tolist()
    test_predictions=[]
    for i in range(len(textqueries)):
        query=textqueries[i]
        filledPrompt=create_incontext_zero_prompt(task, query, context=None, role=role,
output=output)
        response= perform_incontext_classification(model, tokenizer, filledPrompt,  ntokens,
nseq, temp)
        pred=postprocessing(response)
        if pred in["DIRECT", "REPORTED", "JUDGEMENTAL"]:test_predictions.append(pred)
        else: test_predictions.append("DIRECT")
        print("ID: ", idqueries[i], "Predicted: ", pred)


    # Create submission DataFrame
    submission_df = pd.DataFrame({
        'id': idqueries,
        'label': test_predictions,
        "test_case": ["EXIST2024"]*len(test_predictions) })
    submission_df.to_csv('sexism_predictions_task2.csv', index=False)
    print("Prediction TASK2 completed. Results saved to sexism_predictions_task2.csv")


params=dict()
EnTrainTask2, EnDevTask2, EnTestTask2 =reader.get(lang="EN", subset="TRAIN", task="task2"),
reader.get(lang="EN", subset="DEV", task="task2"),  reader.get(lang="EN", subset="TEST",
task="task2")
incontext_zero_pipeline_task2(model, tokenizer, EnDevTask2, EnTestTask2,
output_postprocessing_incontext_zero_s2)
```

EXAMPLE 6: Using zero-shot to sexism categorization

```
def incontext_zero_pipeline_task3(model, tokenizer, devData, testData, postprocessing,
**params):
    role= "in social psychology and linguistics with vast experience analyzing social media
content and discriminative and harmful language"
    task =""" Many facets of a woman’s life may be the focus of sexist attitudes including
domestic and parenting roles, career opportunities, sexual image, and life expectations, to
name a few. According to this, each sexist tweet must be assigned one or more of the five
categories.


    IDEOLOGICAL-AND-INEQUALITY: it includes messages that discredit the feminist movement. It
also includes messages that reject inequality between men and women, or present men as victims
of gender-based oppression.
```

```
   STEREOTYPING-DOMINANCE: it includes messages that express false ideas about women that
suggest they are more suitable or inappropriate for certain tasks, and somehow inferior to men.

   OBJECTIFICATION: it includes messages where women are presented as objects apart from their
dignity and personal aspects. We also include messages that assume or describe certain physical
°qualities that women must have in order to fulfill traditional gender roles

   SEXUAL-VIOLENCE: it includes messages where sexual suggestions, requests or harassment of a
sexual nature (rape or sexual assault) are made.

   MISOGYNY-NON-SEXUAL-VIOLENCE: it includes expressions of hatred and violence towards
women."""

   output= "The output must be a list of the assigned label. Possible Labels: ['IDEOLOGICAL-
INEQUALITY', 'STEREOTYPING-DOMINANCE', 'MISOGYNY-NON-SEXUAL-VIOLENCE', 'OBJECTIFICATION',
'SEXUAL-VIOLENCE']"
   context=None

   ntokens=params.get("max_new_tokens", 32)
   nseq=params.get("num_return_sequences", 1)
   temp=params.get("temperature", 0.7)

   idqueries= devData[0].tolist()
   textqueries= devData[1].tolist()
   labelqueries=devData[2].tolist()

   predictions=[]
   for i in range(len(textqueries)):
       query=textqueries[i]
       filledPrompt=create_incontext_zero_prompt(task, query, context=None, role=role,
output=output)
       response= perform_incontext_classification(model, tokenizer, filledPrompt, ntokens,
nseq, temp)
       pred=postprocessing(response)
       predictions.append(pred)
       print("ID: ", idqueries[i], "Ground Truth:", labelqueries[i], "Predicted: ", pred)
   # Create dev output DataFrame
   dev_df = pd.DataFrame({
       'id': idqueries,
       'label': predictions,
       "tweet": textqueries,
       "test_case": ["EXIST2024"]*len(predictions) })
   dev_df.to_csv('sexism_dev_predictions_task3.csv', index=False)
   print("Evaluation TASK3 completed. Results saved to sexism_dev_predictions_task3.csv")

   #Computing the quality measure for the subtask 1
   icm= ICMWrapper(predictions, labelqueries, multi=True)
   print(f"\nICM : {icm}\n\n")
   idqueries= testData[0].tolist()
   textqueries= testData[1].tolist()
   test_predictions=[]
   for i in range(len(textqueries)):
       query=textqueries[i]
       filledPrompt=create_incontext_zero_prompt(task, query, context=None, role=role,
output=output)
       response= perform_incontext_classification(model, tokenizer, filledPrompt,  ntokens,
nseq, temp)
       pred=postprocessing(response)
       test_predictions.append(pred)
```

```
        print("ID: ", idqueries[i], "Predicted: ", pred)


        # Create submission DataFrame
    submission_df = pd.DataFrame({
        'id': idqueries,
        'label': test_predictions,
        "test_case": ["EXIST2024"]*len(test_predictions) })
    submission_df.to_csv('sexism_predictions_task3.csv', index=False)
    print("Prediction TASK3 completed. Results saved to sexism_predictions_task3.csv")


params=dict()
EnTrainTask3, EnDevTask3, EnTestTask3 =reader.get(lang="EN", subset="TRAIN", task="task3"),
reader.get(lang="EN", subset="DEV", task="task3"),  reader.get(lang="EN", subset="TEST",
task="task3")
incontext_zero_pipeline_task3(model, tokenizer, EnDevTask3, EnTestTask3,
output_postprocessing_incontext_zero_s3)
```

Few-Shot

Few-shot learning is an advanced prompting technique where the model is provided with a small number of example inputs and outputs to help it understand the task before processing the actual input. Indeed, the difference with **zero-shot** relied in which the prompt is informed with few annotated examples from the training data.

Examples 6, 7 and 8 illustrate how to perform few-shot learning for subtasks 1, 2 and 3 in EXIST2025, respectively. In each example the LLaMA 2 model (*meta-llama/Llama-2-7b-chat-hf*) is used as language model. The examples are different with respect to how the prompt is filled in to address the task. The second difference is in how the model's output is parsed to obtain the predicted labels for each tweet. Another difference relies on how the instances are selected to include in the prompt. These examples serve as a starting point for understanding how Large Language Models can be applied to sexism identification tasks through few-shot learning

approaches. Additional work should focus on refining prompt strategies and developing more robust parsing mechanisms to improve classification effectiveness.

EXAMPLE 7: Using few-shot to sexism identification

```
import pandas as pd
from sklearn.metrics import f1_score


def output_postprocessing_incontext_few_s1(output):
   outputp=output.rsplit("\nOutput:", 1)[-1].strip()
   #print(outputp)
   for line in outputp.split('\n'):
       line=line.strip()
       if line!="":
            if line.upper().startswith("YES"): return "YES"
            if line.upper().startswith("NO"): return "NO"
   return "UNK"


#This function aims at selecting the examples used in the in-context learning
# using few-shot. This is a naive approach to inform the prompt with examples in the
training dataset
def sampling_few_instances(trainData, nexamples):
   idqueries= trainData[0].tolist()
   textqueries= trainData[1].tolist()
   labelqueries=trainData[2].tolist()
   df = pd.DataFrame({'id': idqueries, 'tweet': textqueries, 'label': labelqueries})
   samples=stratified_sample(df, 'label', nexamples)
   examples=[]
   for i, row in samples.iterrows():
       examples.append({'input': row['tweet'], 'output': row['label']})
   return examples


def incontext_few_pipeline_task1(model, tokenizer, trainData, devData, testData,
postprocessing,**params):
   role= "in social psychology and linguistics with vast experience analyzing social
media content and discriminative and harmful language"
   task ="""Sexist identification task is a binaty text classification task which
aim at determining
   whether or not a given tweet expresses ideas related to sexism in any of the
three forms: it is sexist itself,
   it describes a sexist situation in which discrimination towards women occurs, or
criticizes a sexist behaviour.
   The tweet is sexist (YES) or describes or criticizes a sexist situation. Not
sexist. The tweet is not sexist (NO),
   nor it describes or criticizes a sexist situation."""
   output= "The output must be YES/NO."
   context=None


   ntokens=params.get("max_new_tokens", 8)
   nseq=params.get("num_return_sequences", 1)
   temp=params.get("temperature", 0.7)
```

```
   idqueries= devData[0].tolist()
   textqueries= devData[1].tolist()
   labelqueries=devData[2].tolist()

   exemplars = sampling_few_instances(trainData, 3)

   predictions=[]
   for i in range(len(textqueries)):
       query=textqueries[i]
       filledPrompt=create_incontext_few_prompt(task, exemplars, query,
context=None, role=role, output=output)
       response= perform_incontext_classification(model, tokenizer, filledPrompt,
ntokens, nseq, temp)
       pred=postprocessing(response)
       if pred in ["YES", "NO"]:predictions.append(pred)
       else: predictions.append("NO")
       print("ID: ", idqueries[i], "Ground Truth:", labelqueries[i], "Predicted: ",
pred)

   # Create dev output DataFrame
   dev_df = pd.DataFrame({'id': idqueries,  'label': predictions, "tweet":
textqueries, "test_case": ["EXIST2024"]*len(predictions) })
   dev_df.to_csv('sexism_dev_predictions_task1_few.csv', index=False)
   print("Prediction TASK1 completed. Results saved to
sexism_dev_predictions_task1_few.csv")

   #Computing the quality measure for the subtask 1
   f1= f1_score(labelqueries, predictions, labels=None, pos_label="YES",
average='binary')
   print(f"\nF1-Score Sexism: {f1}\n\n")

   idqueries= testData[0].tolist()
   textqueries= testData[1].tolist()
   test_predictions=[]
   for i in range(len(textqueries)):
       query=textqueries[i]
       filledPrompt=create_incontext_few_prompt(task, exemplars, query,
context=None, role=role, output=output)
       response= perform_incontext_classification(model, tokenizer, filledPrompt,
ntokens, nseq, temp)
       pred=postprocessing(response)
       if pred in ["YES", "NO"]:test_predictions.append(pred)
       else: test_predictions.append("NO")
       print("ID: ", idqueries[i], "Predicted: ", pred)

       # Create submission DataFrame
   submission_df = pd.DataFrame({
       'id': idqueries,
       'label': test_predictions,
       "test_case": ["EXIST2024"]*len(test_predictions) })
   submission_df.to_csv('sexism_predictions_task1_few.csv', index=False)
   print("Prediction TASK1 completed. Results saved to
sexism_predictions_task1_few.csv")
```

```
params=dict()
EnTrainTask1, EnDevTask1, EnTestTask1 =reader.get(lang="EN", subset="TRAIN", task="task1"),
reader.get(lang="EN", subset="DEV", task="task1"),  reader.get(lang="EN", subset="TEST",
task="task1"),
incontext_few_pipeline_task1(model, tokenizer,EnTrainTask1,  EnDevTask1,
EnTestTask1, output_postprocessing_incontext_few_s1)
```

EXAMPLE 8: Using few-shot to source intention identification

```
import pandas as pd
from sklearn.metrics import f1_score


def output_postprocessing_incontext_few_s2(output):
   outputp=output.rsplit("\nOutput:", 1)[-1].strip()
   for line in outputp.split('\n'):
       line=line.strip()
       if line!="":
            if line.upper().startswith("DIRECT"): return "DIRECT"
            if line.upper().startswith("REPORTED"): return "REPORTED"
            if line.upper().startswith("JUDGEMENTAL"): return "JUDGEMENTAL"
   return "UNK"


#This function aims at selecting the examples used in the in-context learning
# using few-shot. This is a naive approach to inform the prompt with examples in the training
dataset
def sampling_few_instances(trainData, nexamples):
   idqueries= trainData[0].tolist()
   textqueries= trainData[1].tolist()
   labelqueries=trainData[2].tolist()
   df = pd.DataFrame({'id': idqueries, 'tweet': textqueries, 'label': labelqueries})
   samples=stratified_sample(df, 'label', nexamples)
   examples=[]
   for i, row in samples.iterrows():
       examples.append({'input': row['tweet'], 'output': row['label']})
   return examples


def incontext_few_pipeline_task2(model, tokenizer, trainData, devData, testData,
postprocessing, **params):
   role= "in social psychology and linguistics with vast experience analyzing social media
content and discriminative and harmful language"
   task ="""Sexism Source Intention in tweets is a three-class classification task aims to
categorize
   the sexist tweets according to the intention of the author. This distinction allows us to
differentiate
   sexism that is actually taking place online from sexism which is being suffered by women in
other situations
   but that is being reported in social networks with the aim of complaining and fighting
against sexism.
   The task includes the following ternary classification of tweets.


   REPORTED: the intention is to report and share a sexist situation suffered by a woman or
women.
   JUDGEMENTAL the intention is to condemn and criticize sexist situations or behaviours.
   DIRECT: The intention is to write a message that is sexist by itself or incites to be
sexist.
    """
   output= "The output is a single label: REPORTED/JUDGEMENTAL/DIRECT."
```

```
    context=None

    ntokens=params.get("max_new_tokens", 8)
    nseq=params.get("num_return_sequences", 1)
    temp=params.get("temperature", 0.7)

    idqueries= devData[0].tolist()
    textqueries= devData[1].tolist()
    labelqueries=devData[2].tolist()

    exemplars = sampling_few_instances(trainData, 2)

    predictions=[]
    for i in range(len(textqueries)):
        query=textqueries[i]
        filledPrompt=create_incontext_few_prompt(task, exemplars, query, context=None,
role=role, output=output)
        response= perform_incontext_classification(model, tokenizer, filledPrompt, ntokens,
nseq, temp)
        pred=postprocessing(response)
        if pred in ["DIRECT", "REPORTED", "JUDGEMENTAL"]:predictions.append(pred)
        else: predictions.append("DIRECT")
        print("ID: ", idqueries[i],"Ground Truth:", labelqueries[i], "Predicted: ", pred)

    # Create dev output DataFrame
    dev_df = pd.DataFrame({'id': idqueries,  'label': predictions, "tweet": textqueries,
"test_case": ["EXIST2024"]*len(predictions) })
    dev_df.to_csv('sexism_dev_predictions_task2_few.csv', index=False)
    print("Prediction TASK2 completed. Results saved to sexism dev predictions task2 few.csv")

    #Computing the quality measure for the subtask 2
    f1= f1_score(labelqueries, predictions, average='macro')
    print(f"\nMacro average F1-Score Sexism: {f1}\n\n")

    idqueries= testData[0].tolist()
    textqueries= testData[1].tolist()
    test_predictions=[]
    for i in range(len(textqueries)):
        query=textqueries[i]
        filledPrompt=create_incontext_few_prompt(task, exemplars, query, context=None,
role=role, output=output)
        response= perform_incontext_classification(model, tokenizer, filledPrompt,  ntokens,
nseq, temp)
        pred=postprocessing(response)
        if pred in ["DIRECT", "REPORTED", "JUDGEMENTAL"]:test_predictions.append(pred)
        else: test_predictions.append("DIRECT")
        print("ID: ", idqueries[i], "Predicted: ", pred)

        # Create submission DataFrame
    submission_df = pd.DataFrame({
        'id': idqueries,
        'label': test_predictions,
        "test_case": ["EXIST2024"]*len(test_predictions) })
    submission_df.to_csv('sexism_predictions_task2_few.csv', index=False)
    print("Prediction TASK2 completed. Results saved to sexism_predictions_task2_few.csv")

params=dict()
EnTrainTask2, EnDevTask2, EnTestTask2 =reader.get(lang="EN", subset="TRAIN", task="task2"),
reader.get(lang="EN", subset="DEV", task="task2"),  reader.get(lang="EN", subset="TEST",
task="task2")
```

```
incontext_few_pipeline_task2(model, tokenizer,EnTrainTask2,  EnDevTask2, EnTestTask2,
output_postprocessing_incontext_few_s2)
```

EXAMPLE 9: Using few-shot to sexism categorization

```
import pandas as pd
import ast


def output_postprocessing_incontext_zero_s3(output):
   cLabels=['IDEOLOGICAL-INEQUALITY', 'STEREOTYPING-DOMINANCE', 'MISOGYNY-NON-SEXUAL-VIOLENCE',
'OBJECTIFICATION', 'SEXUAL-VIOLENCE']
   labels=[]
   outputp=output.rsplit("\nOutput:", 1)[-1].strip()
   #print(outputp)
   for line in outputp.split('\n'):
       line=line.strip()
       if line!="":
            #print(line)
            if line.upper().startswith("["):
               try:
                   labels = [x.upper() for  x in ast.literal_eval(line) if x.upper() in
cLabels]
                   return labels
               except:
                   labels=[]
   return labels


#This function aims at selecting the examples used in the in-context learning
# using few-shot. This is a naive approach to inform the prompt with examples in the training
dataset
def sampling_few_instances_multilabel(trainData, nexamples):
   idqueries= trainData[0].tolist()
   textqueries= trainData[1].tolist()
   labelqueries=trainData[2].tolist()
   df = pd.DataFrame({'id': idqueries, 'tweet': textqueries, 'label': labelqueries})
   samples= df.sample(n=min(nexamples, len(df)), random_state=1234)
   examples=[]
   for i, row in samples.iterrows():
       examples.append({'input': row['tweet'], 'output': row['label']})
   return examples



def incontext_few_pipeline_task3(model, tokenizer, trainData, devData, testData,
postprocessing, **params):


   role= "in social psychology and linguistics with vast experience analyzing social media
content and discriminative and harmful language"
   task =""" Many facets of a woman’s life may be the focus of sexist attitudes including
domestic and parenting roles, career opportunities, sexual image, and life expectations, to
name a few. According to this, each sexist tweet must be assigned one or more of the five
categories.


   IDEOLOGICAL-AND-INEQUALITY: it includes messages that discredit the feminist movement. It
also includes messages that reject inequality between men and women, or present men as victims
of gender-based oppression.


   STEREOTYPING-DOMINANCE: it includes messages that express false ideas about women that
suggest they are more suitable or inappropriate for certain tasks, and somehow inferior to men.
```

```
   OBJECTIFICATION: it includes messages where women are presented as objects apart from their
dignity and personal aspects. We also include messages that assume or describe certain physical
°qualities that women must have in order to fulfill traditional gender roles

   SEXUAL-VIOLENCE: it includes messages where sexual suggestions, requests or harassment of a
sexual nature (rape or sexual assault) are made.

   MISOGYNY-NON-SEXUAL-VIOLENCE: it includes expressions of hatred and violence towards
women."""

   output= "The output must be a list of the assigned label. Possible Labels: ['IDEOLOGICAL-
INEQUALITY', 'STEREOTYPING-DOMINANCE', 'MISOGYNY-NON-SEXUAL-VIOLENCE', 'OBJECTIFICATION',
'SEXUAL-VIOLENCE']"
   context=None

   ntokens=params.get("max_new_tokens", 48)
   nseq=params.get("num_return_sequences", 1)
   temp=params.get("temperature", 0.4)

   idqueries= devData[0].tolist()
   textqueries= devData[1].tolist()
   labelqueries=devData[2].tolist()

   exemplars = sampling_few_instances_multilabel(trainData, 6)

   predictions=[]
   for i in range(len(textqueries)):
       query=textqueries[i]
       filledPrompt=create_incontext_few_prompt(task, exemplars, query, context=None,
role=role, output=output)
       response= perform_incontext_classification(model, tokenizer, filledPrompt, ntokens,
nseq, temp)
       pred=postprocessing(response)
       predictions.append(pred)
       print("ID: ", idqueries[i],"Ground Truth:", labelqueries[i], "Predicted: ", pred)

   # Create dev output DataFrame
   dev_df = pd.DataFrame({'id': idqueries,  'label': predictions, "tweet": textqueries,
"test_case": ["EXIST2024"]*len(predictions) })
   dev_df.to_csv('sexism_dev_predictions_task3_few.csv', index=False)
   print("Prediction TASK3 completed. Results saved to sexism_dev_predictions_task3_few.csv")

   #Computing the quality measure for the subtask 3
   icm= ICMWrapper(predictions, labelqueries, multi=True)
   print(f"\nICM : {icm}\n\n")

   idqueries= testData[0].tolist()
   textqueries= testData[1].tolist()
   test_predictions=[]
   for i in range(len(textqueries)):
       query=textqueries[i]
       filledPrompt=create_incontext_few_prompt(task, exemplars, query, context=None,
role=role, output=output)
       response= perform_incontext_classification(model, tokenizer, filledPrompt,  ntokens,
nseq, temp)
       pred=postprocessing(response)
       test_predictions.append(pred)
```

```
        print("ID: ", idqueries[i],  "Predicted: ", pred)


    # Create submission DataFrame
    submission_df = pd.DataFrame({
        'id': idqueries,
        'label': test_predictions,
        "test_case": ["EXIST2024"]*len(test_predictions) })
    submission_df.to_csv('sexism_predictions_task3_few.csv', index=False)
    print("Prediction TASK3 completed. Results saved to sexism_predictions_task3_few.csv")


params=dict()
EnTrainTask3, EnDevTask3, EnTestTask3 =reader.get(lang="EN", subset="TRAIN", task="task3"),
reader.get(lang="EN", subset="DEV", task="task3"),  reader.get(lang="EN", subset="TEST",
task="task3")
incontext_few_pipeline_task3(model, tokenizer,EnTrainTask3,  EnDevTask3, EnTestTask3,
output_postprocessing_incontext_zero_s3)
```

Fine-tuning Transformer models requires careful optimization of multiple hyperparameters to achieve optimal performance. While traditional machine learning approaches involve extensive hyperparameter tuning across various model configurations, in-context learning primarily focuses on prompt engineering.

In the context of in-context learning, the prompt itself becomes the critical hyperparameter. Abstract mathematical metrics do not determine its effectiveness but by concrete performance on the specific task at hand. For instance, in sexism identification tasks, the quality of a prompt is empirically evaluated using validation set performance and predefined evaluation measures, such as an F1-Score for the positive class.

The success of a prompt is fundamentally pragmatic: it is judged by its ability to guide the language model toward accurate classifications. A well-designed prompt can significantly impact model performance, translating abstract instructions into precise, task-specific outputs. It is crucial to systematically explore different

prompt designs, analyzing their impact through rigorous evaluation on validation sets.

How Many Examples?

The number of demonstrations does not have to be large. A small number of randomly selected labeled examples used as demonstrations can be sufficient to improve performance over the zero-shot setting. Indeed, the largest performance gains in few-shot prompting tends to come from the first training example, with diminishing returns for subsequent demonstrations. This contrasts with the finetuning of specialized classifier heads where it helps have lots of examples.

Why is it not useful to have more demonstrations? The reason is that the primary benefit in examples is to demonstrate the task to be performed to the LLM and the format of the sequence, not to provide relevant information as to the right answer for any question. In fact, demonstrations that have incorrect answers can still improve a system[6]. Adding too many examples seems to cause the model to overfit to details of the exact examples chosen and generalize poorly.

How to Select Demonstrations?

[6] Webson, A. and E. Pavlick. 2022. Do prompt-based models really understand the meaning of their prompts? NAACL HLT

Demonstrations are created by formatting examples drawn from a labeled training set. There are some heuristics about what makes a good demonstration. For example, using demonstrations that are like the current input seems to improve performance. It can thus be useful to dynamically retrieve demonstrations for each input based on their similarity to the current example (for example, comparing the embedding of the current example with embeddings of each of the training set examples to find the best top-t). But more generally, the best way to select demonstrations from the training set is programmatically: choosing the set of demonstrations that most increases the task performance of the prompt on a validation set.

ACTIVITY

In this lab session, students will engage with the EXIST2025 shared task, focusing on sexism identification in social networks through innovative in-context learning techniques using LLMs. Students can utilize the LLaMA 2 (*meta-llama/Llama-2-7b-chat-hf*) or any other LLM to address **subtask2** in English, employing the two distinct prompting strategies: **zero-shot** and **few-shot**.

Students are challenged to critically design their prompts, carefully considering the output format, number of examples, and sampling strategy. To select the best prompts, it is recommended to perform a prior evaluation with a randomly selected subset of the validation dataset; 100 samples would be sufficient. The evaluation metric will be F1-macro average for subtask 2. The final evaluation will be performed on the **entire validation dataset**.

Recognizing the computational demands of modern language models, students are advised to leverage GPU-equipped hardware to optimize model performance and training efficiency. The deliverable will include not only the source code but also a report that encompasses the prompt engineering strategy, example selection techniques, evaluation results, and challenges encountered. Students are encouraged to supplement their findings with insightful visualizations and analytical tables that demonstrate an understanding of sexism identification through in-context learning with LLMs, especially in the prompting selection.

The Google collaborative environment (Google Colab) can be used to prompt the LLMs:
https://colab.research.google.com/notebooks/welcome.ipynb#scrollTo=5fCEDCU_qrC0

APPENDIX

This appendix provides a comprehensive set of functions used in the preceding examples. These utility functions are critical for implementing the pipelines for zero-shot and few-shot learning.

Wrapper to compute the ICM measure

```
import os, sys, tempfile, pandas as pd
import numpy as np
from pyevall.evaluation import PyEvALLEvaluation
from pyevall.metrics.metricfactory import MetricFactory
from pyevall.reports.reports import PyEvALLReport
from pyevall.utils.utils import PyEvALLUtils


def ICMWrapper(pred, labels, multi=False,ids=None):
   test = PyEvALLEvaluation()
   metrics=[MetricFactory.ICM.value]
   params= dict()
   fillLabel=None
   if multi:
       params[PyEvALLUtils.PARAM_REPORT]="embedded"
       hierarchy={"True":['IDEOLOGICAL-INEQUALITY',  'STEREOTYPING-DOMINANCE',  'MISOGYNY-NON-
SEXUAL-VIOLENCE', 'OBJECTIFICATION', 'SEXUAL-VIOLENCE'],
       "False":[]}
       params[PyEvALLUtils.PARAM_HIERARCHY]=hierarchy
       fillLabel = lambda x: ["False"] if len(x)== 0 else x
   else:
       params[PyEvALLUtils.PARAM_REPORT]="simple"
       fillLabel = lambda x: str(x)


   truth_name, predict_name=None, None
   if ids is None:
       ids=list(range(len(labels)))


   with tempfile.NamedTemporaryFile(mode='w', delete=False, encoding='utf-8') as truth:
       truth_name=truth.name
       truth_df=pd.DataFrame({'test_case': ['EXIST2024']*len(labels),
                       'id': [str(x) for x in ids],
                       'value': [fillLabel(x) for x in labels]})
       if multi==True:
           truth_df=truth_df.astype('object')
       truth.write(truth_df.to_json(orient="records"))


   with  tempfile.NamedTemporaryFile(mode='w', delete=False) as predict:
       predict_name=predict.name
       predict_df=pd.DataFrame({'test_case': ['EXIST2024']*len(pred),
                       'id': [str(x) for x in ids],
                       'value': [fillLabel(x) for x in pred]})
       if multi==True:
           predict_df=predict_df.astype('object')
       predict.write(predict_df.to_json(orient="records"))
```

```
    report = test.evaluate(predict_name, truth_name, metrics, **params)
    os.unlink(truth_name)
    os.unlink(predict_name)


    icm = None
    if 'metrics' in report.report:
        if                    'ICM'                    in                    report.report["metrics"]:
icm=float(report.report["metrics"]['ICM']["results"]["average_per_test_case"])
    return icm
```

Function to submit prompts to an LLM and retrieve generated text

```
def perform_incontext_classification(model, tokenizer, prompt, ntokens=8, nseq=1, temp=0.7):
    inputs = tokenizer(prompt, return_tensors="pt").to(model.device)
    # Generate response
    outputs = model.generate(
        **inputs,
        max_new_tokens=ntokens,
        num_return_sequences=nseq,
        temperature=temp
    )
    response = tokenizer.decode(outputs[0], skip_special_tokens=True)
    return response
```

Function to create the prompts in zero-shot

```
def create_incontext_zero_prompt(task_description, query, context=None, role=None, output=None):
    prompt = f""
    if role!=None:
        prompt+= f"You are an expert {role}.\n\n"
    prompt += f"Task: {task_description}\n\n"
    if context!=None:
         prompt+= f"Context: {context}\n\n"
    if output != None:
         prompt+= f"Output Format: {output} \n\n"
    # Add query
    prompt+=f"Input: {query}\n"
    prompt+=f"Output: "
    return prompt
```

Function to create the prompts in few-shot

```
def create_incontext_few_prompt(task_description, exemplars, query, context=None, role=None,
output=None):
    prompt = f""
    if role!=None:
        prompt+= f"You are an expert {role}.\n\n"
    prompt += f"Task: {task_description}\n\n"
    if context!=None:
         prompt+= f"Context: {context}\n\n"
    prompt += "Examples in Input and Output format:\n\n"
    # Add exemplars
    for exemplar in exemplars:
        prompt += f"Input: {exemplar['input']}\n"
        prompt += f"Output: {exemplar['output']}\n\n"
    if output != None:
         prompt+= f"Output Format: {output} \n\n"
    # Add query
    prompt+=f"Input: {query}\nOutput:"
    return prompt
```

Function to sample the data in a stratified way

```python
def stratified_sample(df, label_column, sample_size=None, sample_frac=None):
    """
    Stratified sampling of a DataFrame based on a label column.

    Args:
        df (pd.DataFrame): The DataFrame to sample.
        label_column (str): The name of the column to stratify by.
        sample_size (int, optional): The desired sample size per stratum.
        sample_frac (float, optional): The fraction of samples to take per stratum.
            Either `sample_size` or `sample_frac` must be provided.

    Returns:
        pd.DataFrame: The stratified sample of the DataFrame.

    Raises:
        ValueError: If neither `sample_size` nor `sample_frac` is provided.
    """

    if sample_size is None and sample_frac is None:
        raise ValueError("Either `sample_size` or `sample_frac` must be provided.")
    # Group the DataFrame by the label column
    grouped = df.groupby(label_column)
    # Determine the sample size or fraction for each group
    if sample_size is not None:
        sample_sizes = {label: sample_size for label in grouped.groups}
    else:
        sample_fracs = {label: sample_frac for label in grouped.groups}
    # Sample each group
    samples = []
    for label, group in grouped:
        if sample_size is not None:
            sample = group.sample(n=sample_sizes[label], random_state=12324)
        else:
            sample = group.sample(frac=sample_fracs[label], random_state=12344)
        samples.append(sample)

    # Concatenate the samples
    stratified_sample = pd.concat(samples, ignore_index=True)
    return stratified_sample
```